FOTA User Manual

AICXTEK Confidential

Documentation Serial Number:	
Documentation Classification:	**Confidential**
Documentation Owner:	**AICXTEK**
Documentation Version:	**1.3**

Author:	**R&D Center**
Date:	**2024.4**

版本更新记录			
版本号	日期	作者	版本更新描述
1.0	2023-07-31	R&D Center	文档建立
1.1	2023-09-01	R&D Center	更新差分包制作章节
1.2	2023-11-21	R&D Center	增加FOTA API的示例代码
1.3	2024-04-02	R&D Center	增加MCU方式的接口说明和删除差分包的类型描述

AICXTEK Confidential

本文档（包括任何附件，及其不时的修订）为归芯科技（深圳）有限公司及其关联公司（合称“归芯”）的保密信息。归芯有权不时修订本文档。本文档如有变更，恕不另行通知。
除非已另行获得归芯的明确授权，请勿向第三方透露本文档的任何内容，请勿将本文档的任何内容以任何方式进行复制、发布、修改、传播、分发、翻印、拆解、反向工程、反编译或者为任何目的使用。
归芯始终持有本文档的全部所有权和著作权、专利权、商业秘密、商标权和其他知识产权。除非归芯另行明确授予有关权利，本文档不提供有关上述知识产权的任何权利。
归芯按“现状”提供本文档及归芯产品或服务。归芯未对本文档中的任何内容或本文档涉及的归芯的产品或服务作出任何明示或暗示的陈述、保证或承诺，包括但不限于任何针对特定目的的适销性或适用性，准确性或完整性，或不侵权。
在法律允许的范围内，归芯不对与本文档有关或因本文档内容造成的任何损失承担责任。在任何情况下，归芯都不对因使用本文档造成的或与之有关的任何间接、偶然的、附带或惩罚性损失，或任何利润、收入的损失承担责任。

This document (including any attachment, as amended from time to time) is confidential information of AICXTEK technology co.,ltd. and its affiliates (collectively referred to as “AICXTEK”). AICXTEK reserves the right to amend this document from time to time. Any changes to this document will not be notified.
Unless otherwise expressly authorized by AICXTEK, you should not disclose any of this document to any third parties. Nor should you copy, publish, modify, broadcast, distribute, photocopy, disassemble, reverse engineer, or decompile any of this document or use it for any purpose.
AICXTEK retains all rights of ownership and copyright, patent rights, trade secrets, trademark rights and other intellectual property rights pertaining to this document. Except for rights otherwise expressly granted by AICXTEK, this document does not provide any rights to any of the above intellectual property rights.
AICXTEK provides this document and AICXTEK’s products or services on a “as-is” basis. AICXTEK does not provide any representations or warranties on any of this document, or AICXTEK’s products or services, whether express or implied, including without limitation, merchantability, fitness for a particular purpose, accuracy or completeness, or non-infringement.
To the extent permitted by law, AICXTEK is not liable for any loss relating to or arising from this document. AICXTEK is not in any event be liable for any indirect, incidental or punitive damages, or loss of profits or revenue.

AICXTEK Confidential

目录

目录 1
1. 方案整体介绍 2
2. 接口说明 2
2.1 API方式 2
2.2 AT方式 3
2.3 MCU方式 5
3. 差分包制作 6
3.1 差分包的制作 6
3.2 差分包的上传 7
3.3 差分包的升级 7
4. 集成说明 7
5. 附录 9

AICXTEK Confidential

1. 方案整体介绍

FOTA功能用于在线升级设备固件，升级主要包括两部分，一部分是通过无线网络下载固件差分包，一部分是下载成功后进行升级。固件的下载目前支持HTTP协议，其他协议如HTTPS/FTP等可根据客户需求支持，下载支持断点续传。

传统的FOTA方案，需要预留专门的flash分区用来存放差分包以及进行升级期间的掉电保护，但对于物联网领域，Flash空间都非常紧张，无法预留专门的FOTA分区，需要复用现有的Flash分区，存放差分包，以及用于掉电保护。

2. 接口说明

2.1 API方式

- fota_start

传入服务器url、用户名、密码、端口号、urc上报使用的串口号及波特率等。

示例：

```
struct fota_http_input {
   char url[FOTA_URL_LEN]; //差分包url
   char user_name[FOTA_USER_NAME_LEN]; //用户名
   char password[FOTA_PWD_LEN]; //密码
   u32 port; //http端口号
};

struct fota_input {
   struct fota_http_input http;
   char crt_url[FOTA_URL_LEN]; //https证书url，暂未使用
   u8 *crt; //https证书，暂未使用
   u32 crt_len; //https证书长度，暂未使用

   u32 serial_port_id; //用于升级上报的串口号
   u32 serial_port_baud; //用于升级上报的串口波特率
   u32 dl_urc; //下载进度提示最大次数
   u32 upgrd_mode; //升级模式，暂未使用
   u32 upgrd_urc; //升级进度提示最大次数
   u32 upgrd_type; //升级类型，全部差分包升级或者只升级特定分区(如app分区)
   u32 reserved; //保留
};

static int apifota_start_test(void)
{
        struct fota_input fota_input;
        u8 url[]="http:// xx.xx.xx.xx /doc/test_fota/gx316h_h1_old_to_new.diff";
        u8 user_name[]="";
        u8 password[]="";

        strncpy(fota_input.http.url, url, strlen(url));
        fota_input.http.port=80;
        strncpy(fota_input.http.user_name, user_name, strlen(user_name));
        strncpy(fota_input.http.password, password, strlen(password));
```

AICXTEK Confidential

```
        fota_input.serial_port_id=conn_get_at_com_number(fota_input.serial_port_id);
        fota_input.serial_port_baud=conn_cmd_get_baud_rate(CONN_CMD_AT,
                                                    fota_input.serial_port_id);
        fota_input.upgrd_mode=0;
        fota_input.dl_urc=10;
        fota_input.upgrd_urc=10;

        fota_start(&fota_input);
        return 0;
}
```

- fota_stop
  停止fota服务
- fota_notify_register
  开机时调用，注册客户自己实现的cb用于urc上报

示例:

```
static void apifota_callback(void *user_data, int event, u32 data1, u32 data2)
{
   FLOGI("apifota_callback test!!!\n");
}
static int apifota_callback_init(void)
{
    if(AIC_SUCCESS != fota_notify_register(apifota_callback, 0,
                                           FOTA_START_BY_APP))
    {
       FASSERT(0);
       return -1;
    }
        return 0;
}
AIC_BASIC_APP_P2_INIT_EXPORT(apifota_callback_init);
```

- fota_notify_unregister
  取消客户cb的注册
- fota_get_stage
  获取当前fota升级进行到哪个阶段
- fota_get_input_param
  获取客户传入的fota input parameter
- fota_get_download_size
  获取当前正在下载文件的total size，以及已经下载的size

具体的API及相关结构体，请参考：/aicos/components/include/aic_fota.h

2.2 AT方式

相关命令共有两个：

1) 设置HTTPS协议证书：AT+FOTACRT

命令：

AT+FOTACRT=?

返回：
AT+FOTACRT=<size>

命令：
AT+FOTACRT=<size>
返回：
FOTACRT START
OK
参数：
size：证书大小，以字节为单位。

说明：

1）只有使用HTTPS下载时才需要设置证书

2）用AT+FOTACRT=<size>设置证书的时候，如果成功，则返回FOTACRT START，这个时候AT命令已自动从命令（CMD）模式切到数据（DATA）模式，然后就可以发送HTTPS证书内容了；内容发送完毕，需要发送“+++”，设备收到后把AT命令再切回命令模式。发送的证书内容长度如果小于size，则视为证书设置错误；如果大于size，则自动截取前面size个字节为证书有效数据。

2) 启动FOTA：AT+FOTA

命令：
AT+FOTA=?
返回：
+FOTA=<url>,<port>,<user_name>,<pwd>,(0,1),(0,5-100),(0,5-100)

命令：
AT+FOTA?
返回：
+FOTA:STATE NONE

OK

命令：
AT+FOTA=<url>,<port>,<user_name>,<pwd>,<reserve>,<dl_step>,<upd_step>
返回：
OK

参数：

<url>：服务器差分包的文件路径；
<port>：端口号。HTTPS默认为443，根据服务器配置设置；
<user_name>：登录用户名，根据服务器配置设置；
<pwd>：登录密码；
<reserve>：保留，暂时无意义；

<dl_step>：下载进度提示最大次数，如果为0不提示。比如设置20，则下载时最多提示20次。

<upd_step>：升级进度提示最大次数。

比如：AT+FOTA=”http://xx.xx.xx.xx/fota_test/gx316ch0.basic_d”,80,””,””,0,10,10

2.3 MCU方式

按照以下顺序执行：

1）aic_fota_start (u32 pac_size)

功能：

启动fota升级，并把差分包大小告诉AIC，让AIC准备接收差分包（申请内存、线程、信号量等资源）。

参数：

pac_size：差分包的大小。

返回值：

0：成功。

其它：失败。

2）aic_fota_trans_data (void *data, u32 size)

功能：

传输差分包，可以一次传完或分多次传输。

参数：

data：此次所传数据的地址。

size：此次所传数据的大小。

返回值：

0：成功。

其它：失败。

3）aic_fota_wait_update_result (int timout)

功能：

等待升级完成。

参数：

timeout：最大等待时间，单位是秒，建议3分钟左右比较安全。

返回值：

AIC_FOTA_UPDATE_COMPLETE：升级成功。

AIC_FOTA_PAC_TRNS_TIMEOUT：传输差分包超时。

AIC_FOTA_PAC_VER_NOT_MATCH：差分包和AIC版本不匹配。

AIC_FOTA_PAC_CHECK_FAIL：校验失败。

AIC_FOTA_UPDATE_TIMEOUT：升级超时，需要重启AIC重试，或者增加超时时间重试。

4）aic_fota_stop

功能：

如果正在升级，则结束当前升级；

如果已经升级成功，不影响升级结果；

AICXTEK Confidential

第三步执行完，无论升级成功或失败，都必须调用此接口释放aic_fota_start所申请的资源。

备注：

为方便客户更容易获取版本号，新增2个API，仅供参考。

- aic_fota_get_aic_remote_ver_num

  功能：

  获取AIC的modem侧的版本号，简称remote version。

- aic_fota_get_aic_local_ver_num

  功能：

  获取AIC的版本在MCU侧的版本号，简称local version。

在MCU侧代码升级完之后，通过调用以上2个API获取2个版本号进行比较，如果local version大于remote version，说明modem侧需要升级；

如果客户有自己的版本管理方式，可以用自己的API判断。

示例：

参考MCU SDK目录*\apps\common\cat1\aicxtek\aic_srv\sample\中的fota.c文件。

具体的API及相关升级结果的状态，请参考目录*\apps\common\cat1\aicxtek\aic_srv\inc\中的aic_fota.h文件。

3. 差分包制作

3.1 差分包的制作

运行gxdiff.exe

对应不同board，需要配置下Setting.ini，如下是.ini文件的配置说明：

```
[pkg]
old_pkg = .\gx316l_h2_old.pkg
new_pkg = .\gx316l_h2_new.pkg
diff_name = gx316l_h2_old_to_new

[scheme]
need_update_romfs = 0
need_full_diff = 1
fota_bak = ap_extend
only_has_basic = 0
```

参数说明：

[pkg]:

old_pkg:填写old pkg路径。

new_pkg:填写new pkg路径。

diff_name:合并后的差分包的名称。

[scheme]

need_update_romfs:配置是否需要制作romfs分区差分。

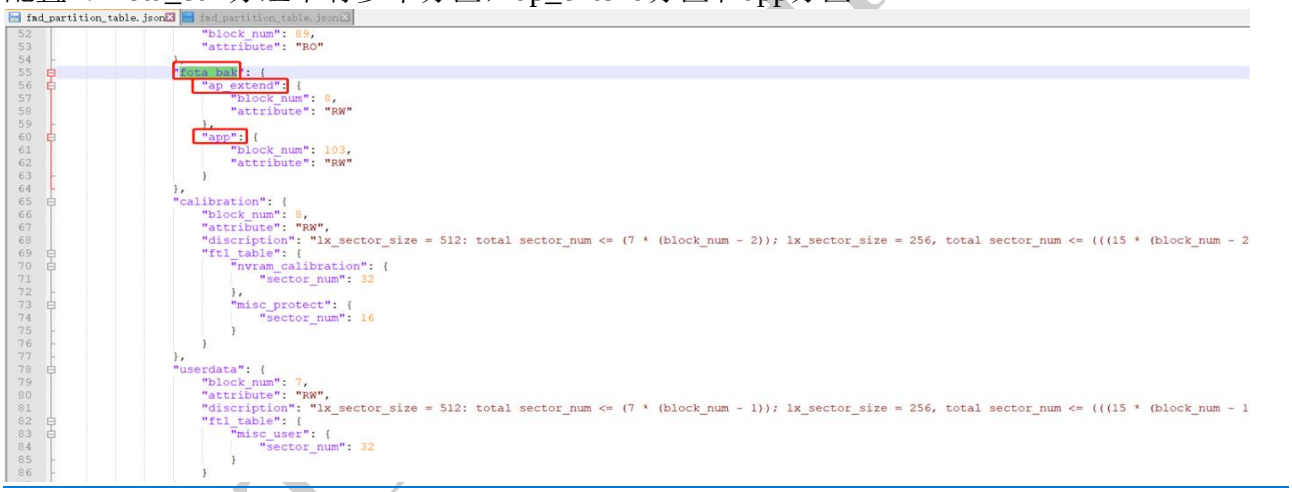
AICXTEK Confidential

need_full_diff:配置是否需要制作整包。
fota_bak:配置fota_bak下所有的分区名，要和如下json文件一致。
配置1：fota_bak分组中只有一个ap_extend分区。

fmd_partition_table.json

```
"misc": {
    "block_num": 3,
    "attribute": "RO"
},
"ap": {
    "block_num": 232,
    "attribute": "RO"
},
"fota_bak": {
    "ap_extend": {
        "block_num": 219,
        "attribute": "RW"
    }
},
```

配置2：fota_bak分组中有多个分区，ap_extend分区和app分区。



only_has_basic:配置是否只有basic分区。

3.2 差分包的上传

根据上述场景，修改差分工具的配置文件，并生成对应的差分包，并上传到服务器。

3.3 差分包的升级

通过AT或API方式传入FOTA服务器url时，需传入差分包的文件名，比如：

AT+FOTA=”http://xx.xx.xx.xx/fota_test/gx316ch0.diff”,80,””,””,0,10,10

4. 集成说明

在集成新FOTA方案时，需要调整ld文件及分区表，新增extend分区。且该分区size不能太小，不能小于允许的最大差分包的size + 掉电保护单元size。在4+2上，最大差分包size是200KB左右，掉电保护单元是244KB，具体size会根据实际Flash剩余空间有所不同。

另外，在编译时或制作差分脚本时，会自动做上述检测，如不满足条件会报错。

- extend的拆分原则

非归芯的代码尽量都放到EXTEND IMAGE里面。

- extend的调用规则

为了方便check，basic 模式下会把extend image 的可执行权限去掉，这样一旦有误调用会出发Hard Fault异常。

遵循方法：

1. Basic image 里面的代码尽量不调用或访问Extend image 里面的代码（包含rodata）。如果有些正常开机确实要调用的代码请用下面这个函数来保证basic模式下不访问。

```
/* basic mode can't call them. */
if (!aic_boot_mode_is_basic()) {
    aic_cali_data_register_hook(aic_md_lock_callback, NULL);
    aic_cali_register_hook(aic_md_cali_callback, NULL);
}
```

2. 对于AutoInit 的接口，请使用下面的系列接口

AIC_EXTIMG_XXX _INIT_EXPORT

详见aic_init.h。

- basic/extend的拆分方法

拆分的方法主要靠修改连接描述文件aic_board.ld 文件来实现。新增加的代码都将自动默认放入extend image。

在对应board目录下，比如gx316c_h0，aicos\projects\genesis\ap\gx316c_h0下的aic_board.ld文件中，有2个SECTIONS，第一个是basic image的SECTION，第二个是extend image的SECTION。每个image中主要用.xxx_text 和.xxx_rodata 2段来分别描述代码和rodata的在flash中的存放。因为gcc 的连接脚本 .ld文件在链接时是按照从上到下的先后顺序来执行的。 所以extend_image 里面 .extend_text 和 .extend_rodata 的描述分别用的 *(.text .text.*) 和 *(.rodata .rodata.*)， 意思就是在此之前的basic image 中描述的链接信息没有包含到的所有内容都会放到extend_image。

放入basic image的方法：

- 特定段

  比如写了些函数及一些数组常量，

  代码用AIC_SECTION(".test_code ") 来修饰这些函数和数组。

  在aic_board.ld中的.basic_text 段落加入下面的语句

  *(.test_code)

  在aic_board.ld中的.basic_rodata 段落中加入下面的语句

  *(.test_code)

- 整个.lib

  在aic_board.ld中的.basic_text 段落加入下面的语句

  \libtest.a:(.text .text.*)

AICXTEK Confidential

在aic_board.ld中的.baisc_rodata 段落中也加入下面的语句
\libtest.a: (.rodata .rodata.*)

- **整个.o**

在aic_board.ld中的.basic_text 段落加入下面的语句
test.*(.text .text.*)
在aic_board.ld中的 .basic_rodata 段落中也加入下面的语句
test.* (.rodata .rodata.*)
也可以用test_*.* 表示以test_开头的所有文件

- .lib中排除某些.o文件

在aic_board.ld中的.basic_text 段落加入下面的语句
EXCLUDE_FILE(test_lcd.* test_camera.*)
\libtest.a:(.text .text.*)
在aic_board.ld中的.basic_rodata 段落加入下面的语句
(test_lcd.* test_camera.*)
\libtest.a: (.rodata .rodata.*)

- **特殊代码放到 psram上**

按照上面章节说的类似的方法把对应描述语句分别加入aic_board.ld中的.basic_psram_text 和 .basic_psram_rodata 段落 即可。

- 客户自定义分区

- 如果该分区只是用于数据存储，不用于下载bin文件，则对FOTA方案没有影响，只需要确保MPU配置以及分区表配置正确即可。
- 如果该分区会用于下载bin文件，则需要结合上述章节进行适配。要么将该分区的内容编译到extend分区中，要么编译到ap分区中，具体情况再针对性讨论。

5. 附录

包含某一个board 的完整的aic_board.ld 文件及简单注释。

```
#include <sys_mem_config.h> //此文件中用到的一些宏都在此头文件或此文件包含的头文件中定义

/* 下面定义了所有code（包含RW，heap等）用到的所有段的区域 */
MEMORY
{
   VECTOR_REGION(rx) : ORIGIN = VECTOR_REGION_BASE, LENGTH = VECTOR_REGION_SIZE //中段向量及sleep代码（aon iram）
   STACK_REGION(rw) : ORIGIN = ALL_STACK_BASE, LENGTH = ALL_STACK_SIZE  //系统栈（aon iram）
   FLASH_BASIC (rx) : ORIGIN = FLASH0_AP_BASE, LENGTH = FLASH0_AP_SIZE //basic image (flash)
   FLASH_EXTEND (rx) : ORIGIN = FLASH0_AP_EXTEND_BASE, LENGTH = FLASH0_AP_EXTEND_SIZE //extend image (flash)
   PSRAM_RO (rx) : ORIGIN = CODE_REGION_BASE, LENGTH = CODE_REGION_SIZE // psram code
   PSRAM_RW (rw) : ORIGIN = RW_REGION_BASE, LENGTH = RW_REGION_SIZE // rw (psram)
   PSRAM_HEAP (rw) : ORIGIN = HEAP_REGION_BASE, LENGTH = HEAP_REGION_SIZE // heap (psram)
   MTB_REGION (rw) : ORIGIN = 0x54000600, LENGTH = 0x200 //flash 驱动专用，不用关注（mtb）
}

/* basic image section */
SECTIONS
{
   .basic_init : //第一段放 init 代码， 也是大系统的入口， 不能改变。
   {
      __init_start__ = .;
      *(.init .init.*)
      __init_end__ = .;
   } > FLASH_BASIC
```

AICXTEK Confidential

```
.basic_text : //basic image 的code（不含数据）
{
    __basic_text_start__ = .;
    *(.glue_7t .glue_7 .gnu.linkonce.t.* .gcc_except_table .gnu.sgstubs .rel.*)
    *\libnosys.a:* (.text .text.*)
    *\libm.a:*(.text .text.*)
    *\libgcc.a:*(.text .text.*)
    *\libc_nano.a:*(.text .text.*)
    *\libc.a:*(.text .text.*)
    *\libaic_osa.a:*(.text .text.*)

    EXCLUDE_FILE(*_at.*)
    *\libaic_fs.a:*(.text .text.*)

    *\liblos_tlsf.a:*(.text .text.*)
    *\libthreadx.a:*(.text .text.*)
    *\liblevelx.a:*(.text .text.*)
    *\libfilex.a:*(.text .text.*)
    *\libnetxduo.a:*(.text .text.*)
    *\libmbedcrypto.a:*(.text .text.*)
    *\libmbedx509.a:*(.text .text.*)
    *\libmbedtls.a:*(.text .text.*)
    *\liblvgl.a:*(.text .text.*)
    *\libboard.a:*(.text .text.*)

    EXCLUDE_FILE(snor.* gigadevice.* winbond.* xmc.* puya.* ux_dcd* usb*)
    *\libdrivers.a:*(.text .text.*)

    EXCLUDE_FILE(qspi.* psram.* sleep_manage.* ap_slp.* wake_lock.*
            busmon_* dump_addr.* breakpoint.* perf_*)
    *\libcs_drivers.a:*(.text .text.*)

    EXCLUDE_FILE(aic_cmd_exe.* aic_console.* imei_diag.* imei_at.* mac.*
            sn.* aic_cali_data_cmd.* aic_factory_cmd.*)
    *\libaic_components.a:*(.text .text.*)

    EXCLUDE_FILE(aic_mcx.* alog_store_mgr.* aic_network_pcap.*)
    *\libcs_components.a:*(.text .text.*)

    /*
     * customer can add basic bsp driver code here, like below:
     * lcd_1602_driver*(.text .text.*)
     */

    __basic_text_end__ = .;
} > FLASH_BASIC

.basic_rodata : //basic image 的ro data
{
    __basic_rodata_start__ = .;
    *(.gnu.linkonce.r.*)
    *\libnosys.a:*(.rodata .rodata.*)
    *\libm.a:*(.rodata .rodata.*)
    *\libgcc.a:*(.rodata .rodata.*)
    *\libc_nano.a:*(.rodata .rodata.*)
    *\libc.a:*(.rodata .rodata.*)
    *\libaic_osa.a:*(.rodata .rodata.*)

    EXCLUDE_FILE(*_at.*)
    *\libaic_fs.a:*(.rodata .rodata.*)

    *\liblos_tlsf.a:*(.rodata .rodata.*)
    *\libthreadx.a:*(.rodata .rodata.*)
    *\liblevelx.a:*(.rodata .rodata.*)
    *\libfilex.a:*(.rodata .rodata.*)
    *\libnetxduo.a:*(.rodata .rodata.*)
    *\libmbedcrypto.a:*(.rodata .rodata.*)
    *\libmbedx509.a:*(.rodata .rodata.*)
    *\libmbedtls.a:*(.rodata .rodata.*)
    *\liblvgl.a:*(.rodata .rodata.*)
    *\libboard.a:*(.rodata .rodata.*)

    EXCLUDE_FILE(snor.* gigadevice.* winbond.* xmc.* puya.* ux_dcd* usb*)
```

AICXTEK Confidential

```
    *\libdrivers.a:*(.rodata .rodata.*)

    EXCLUDE_FILE(qspi.* psram.* sleep_manage.* ap_slp.* wake_lock.*
                busmon_* dump_addr.* breakpoint.* perf_*)
    *\libcs_drivers.a:*(.rodata .rodata.*)

    EXCLUDE_FILE(aic_cmd_exe.* aic_console.* imei_diag.* imei_at.* mac.*
                sn.* aic_cali_data_cmd.* aic_factory_cmd.*)
    *\libaic_components.a:*(.rodata .rodata.*)

    EXCLUDE_FILE(aic_mcx.* alog_store_mgr.* aic_network_pcap.*)
    *\libcs_components.a:*(.rodata .rodata.*)

    /*
     * customer can add basic bsp driver code here, like below:
     * lcd_1602_driver*(.rodata .rodata.*)
     */

    KEEP (*(SORT(.auto_init*)))// keep的目的是保留那些没有被显示调用的代码，让它们不被编译器优化掉
    KEEP (*(SORT(.aiccmd*)))
    __basic_rodata_end__ = .;
  } > FLASH_BASIC

  .basic_ARM.exidx : //不用关注
  {
    __exidx_start = .;
    *(.ARM.exidx* .gnu.linkonce.armexidx.*)
    __exidx_end = .;
  } > FLASH_BASIC

  __vec_load_start__ = ALIGN(__exidx_end, 4); //中的向量代码
  .basic_vectors : AT(__vec_load_start__)
  {
    __vectors_start__ = .;
    *(.vectors .vectors.*)
    __vectors_end__ = .;
  } > VECTOR_REGION
  __vec_load_end__ = __vec_load_start__ + SIZEOF(.basic_vectors);

  __idle_load_start__ = ALIGN(__vec_load_end__, 4); //sleep代码
  .basic_idle : AT(__idle_load_start__)
  {
    __idle_start__ = .;
    *(.idle_text .idle_text.* .idle_data .idle_data.*)
    psram.* (.text .text.* .rodata .rodata.* .data .data.*)
    * (.qspi_code .qspi_rwdata)
    __idle_end__ = .;
  } > VECTOR_REGION
  __idle_load_end__ = __idle_load_start__ + SIZEOF(.basic_idle);

  __mtb_load_start__ = ALIGN(__idle_load_end__, 4); //不用关注
  .basic_mtb : AT(__mtb_load_start__)
  {
    __mtb_start__ = .;
    * (.mtb_code)
    __mtb_end__ = .;
  } > MTB_REGION
  __mtb_load_end__ = __mtb_load_start__ + SIZEOF(.basic_mtb);

  __psram_text_load_start__ = ALIGN(__mtb_load_end__ , 4);//PSRAM代码，目前放的是不能在flash上执行的代码，主要是flash本身的驱动和flash
还没有ready时候就需要执行的一些代码
  .basic_psram_text : AT(__psram_text_load_start__)
  {
    __psram_text_start__ = .;
    snor* (.text .text.*)
    qspi.* (.text .text.*)
    gigadevice.* (.text .text.*)
    winbond.* (.text .text.*)
    xmc.* (.text .text.*)
    puya.* (.text .text.*)
    aic_mcx.* (.text .text.*)
    * (.mpu_text)
    sleep_manage.* (.text .text.*)
```

AICXTEK Confidential

```
    ap_slp.*  (.text .text.*)
    wake_lock.*  (.text .text.*)
    * (.timer_code)
    * (.spinlock_code)
    __psram_text_end__ = .;
  } > PSRAM_RO
  __psram_text_load_end__ = __psram_text_load_start__ + SIZEOF(.basic_psram_text);

  __psram_rodata_load_start__ = ALIGN(__psram_text_load_end__ , 4); //PSRAM RODATA，同上
  .basic_psram_rodata : AT(__psram_rodata_load_start__)
  {
    __psram_rodata_start__ = .;
    snor* (.rodata .rodata.*)
    qspi.* (.rodata .rodata.*)
    gigadevice.* (.rodata .rodata.*)
    winbond.* (.rodata .rodata.*)
    xmc.* (.rodata .rodata.*)
    puya.* (.rodata .rodata.*)
    aic_mcx.* (.rodata .rodata.*)
    sleep_manage.* (.rodata .rodata.*)
    ap_slp.*  (.rodata .rodata.*)
    wake_lock.*  (.rodata .rodata.*)
    __psram_rodata_end__ = .;
  } > PSRAM_RO
  __psram_rodata_load_end__ = __psram_rodata_load_start__ + SIZEOF(.basic_psram_rodata);

  .basic_heap (NOLOAD) : //heap, no load
  {
    __heap_start__ = .;
    *(.sys_heap)
    __heap_end__ = ALIGN(4);
  } > PSRAM_HEAP

  __data_load_start__ = ALIGN(__psram_rodata_load_end__, 4);// RW
  .basic_data : AT(__data_load_start__)
  {
    __data_start__ = .;
    *(.data .data.* *.data .gnu.linkonce.d.* .gnu.sgstubs)
    __data_end__ = .;
  } > PSRAM_RW
  __data_load_end__ = __data_load_start__ + SIZEOF(.basic_data);

  .basic_bss (NOLOAD) : //bss, no load
  {
    __bss_start__ = .;
    *(.trace_buffer .bss .bss.* .gnu.linkonce.b.*) *(COMMON)
    __bss_end__ = .;
  } > PSRAM_RW

  .basic_stack (NOLOAD) : //sys stack, 系统初始化，中断处理，异常处理的栈
  {
    __stack_start__ = .;
    *(.sys_stack)
    __stack_end__ = ALIGN(4);
  } > STACK_REGION

  __RAM_segment_used_end__ = ALIGN(__bss_end__ , 4);
}

// extend image SECTION
SECTIONS
{
  .extend_head :// extend image header, 512 byte, reserve for save fota diff package info.
  {
    KEEP (*(SORT(.extend_header)))
  } > FLASH_EXTEND

  .extend_text :
  {
    __extend_text_start__ = .;
    *(.text .text.* )
    __extend_text_end__ = .;
  } > FLASH_EXTEND
```

AICXTEK Confidential

```
    .extend_rodata :
    {
      __extend_rodata_start__ = .;
     *(.rodata .rodata.*)
      __extend_rodata_end__ = .;
    } > FLASH_EXTEND
}
```

AICXTEK Confidential